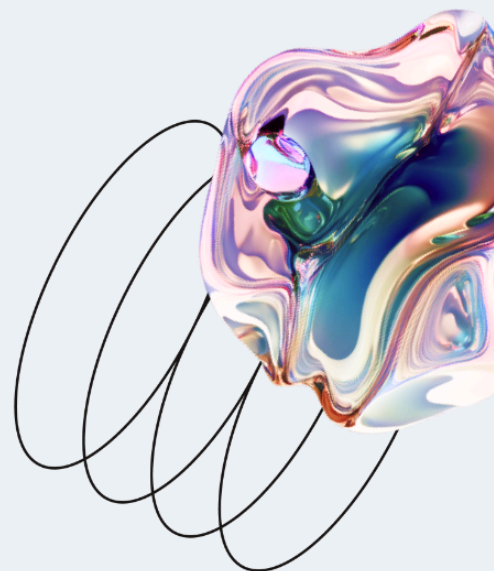




Безопасная разработка приложений

Основы информационной безопасности



Оглавление

Введение.....	4
Словарь терминов.....	4
Модель жизненного цикла безопасности приложений.....	5
Принципы безопасной разработки приложений.....	6
Отличительная особенность моделирования угроз при разработке ПО.....	8
Популярные best practices и framework-и.....	8
ISO/IEC 27034 Application Security и ГОСТ.....	8
NIST SSDF.....	12
OWASP CLASP и SSDF.....	13
Рекомендации по безопасному написанию кода.....	14
Валидация входных данных (Input validation).....	14
Кодирование выходных данных (Output encoding).....	15
Аутентификация и управление паролями (Authentication and password management).....	15
Управление сессиями (Session management).....	17
Управление доступом (Access control).....	18
Криптографическая защита (Cryptographic practices).....	19
Обработка ошибок и регистрация событий (Error handling and logging).....	20
Защита данных (Data protection).....	21
Защита коммуникаций (Communication security).....	21
Безопасная конфигурация системы (System configuration).....	22
Безопасность баз данных (Database security).....	23

Безопасность файлов (File management)	23
Безопасность данных в памяти (Memory management)	24
Общие практики кодирования (General coding practices)	24
Основные способы верификации требований к безопасности для приложений.....	25
Безопасность в цикле CI/CD – DevSecOps.....	26
Выводы.....	27
Дополнительные материалы.....	27
Использованная литература.....	28

Введение

Сегодня мы продолжим курс, посвящённый основам обеспечения информационной безопасности (ИБ), а текущее занятие – безопасная разработка приложений.

На этой лекции вы узнаете:

- Что такое эталонная модель жизненного цикла безопасности приложений (Security Development Lifecycle) и зачем она нужна?
- Какие ключевые принципы **безопасной разработки** приложений?
- Какие есть популярные best practices и framework по безопасной разработке приложений?
- Какие есть ключевые рекомендации по безопасному написанию кода?
- Какие есть основные способы верификации требований к безопасности для приложений?
- Как возникает DevSecOps?

Словарь терминов

Программное обеспечение (приложение) – совокупность программ системы обработки информации и программных документов, необходимых для эксплуатации этих программ ([ГОСТ Р 56939-2016](#)).

Безопасное программное обеспечение – программное обеспечение, разработанное с использованием совокупности мер, направленных на предотвращение появления и устранение уязвимостей программы ([ГОСТ Р 56939-2016](#)).

Валидация (validation) – подтверждение посредством представления объективных свидетельств того, что требования, предназначенные для конкретного применения, выполнены ([ГОСТ Р ИСО/МЭК 27034-1-2014](#)).

Верификация (verification) – подтверждение посредством представления объективных свидетельств того, что установленные требования были выполнены ([ГОСТ Р ИСО/МЭК 27034-1-2014](#)).



В части понятий *валидация* и *верификация* часто возникает путаница.

Если упросить определения, то валидация отвечает на вопрос «Правильное ли приложение создано?» (т. е. предназначенное для использования / применения в конкретных условиях), а верификация –

«Правильно ли создано приложение?» (т.е. соблюдены ли установленные к нему требования).

Валидация чаще всего применима именно к данным (например, входным данным), а верификация к ПО (коду).

Модель жизненного цикла безопасности приложений

Стоит начать с того, что существуют различные концептуальные модели жизненного цикла разработки программного обеспечения (ПО), например: каскадная (waterfall), спиральная (spiral), гибкая (agile) и др., которые ориентированы и учитывают различные аспекты и особенности разработки ПО. Но, к сожалению, лишь в немногих из этих моделей безопасность ПО рассматривается в явном виде, поэтому методы разработки безопасного ПО обычно приходится дополнять и интегрировать в каждую такую модель.

Обсуждая жизненный цикл разработки ПО, используется понятие Software Development Life Cycle (SDLC) как формальная или неформальная методология дизайна, создания и поддержки ПО. По аналогии возникло понятие Security Development Lifecycle (SDL) или Secure Software Development Life Cycle (SSDLC), т.е. методология дизайна, создания и поддержки безопасного ПО. Далее, для краткости будем использовать аббревиатуру SDL.

При этом, к сожалению, в области ИБ нет единой модели SDL, так же как и в SDLC, в зависимости от акцентов, расставленных автором цикла, может меняться область его применения и количество шагов в цикле.

Например, наиболее популярный SDL от Корпорации Microsoft [1], включает в себя 7 шагов и затрагивает не только само ПО, но и обучение разработчиков, а также реагирование на инциденты, возникающие уже после выпуска ПО:

- обучение разработчиков;
- формирование требований к разрабатываемому ПО;
- проектирование (дизайн) ПО;
- непосредственно разработка ПО (кодирование);
- верификация разработанного ПО;
- выпуск программного продукта.

- своевременное реагирование на возникающие инциденты и проблемы по линии безопасности при использовании выпущенного программного продукта;

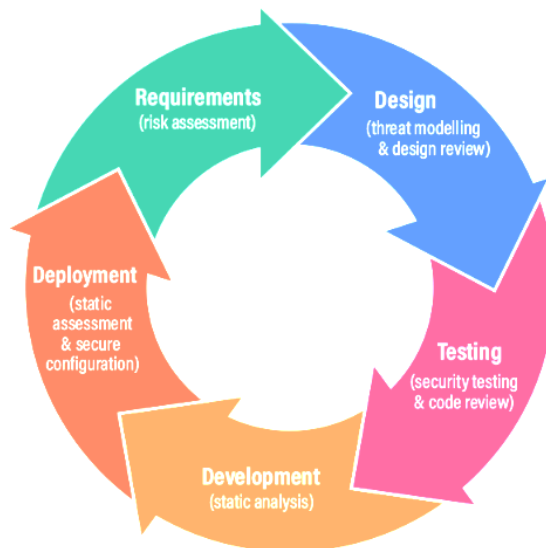


Институт NIST увеличил SDL до 4 шагов и сформировал Secure Software Development Framework (SSDF) [2]. И в рамках данного framework, явно выделил защиту самого ПО – «Protect Software», включая репозитории, архивы, а также предоставление для пользователей информации для проверки целостности приобретаемого (скачиваемого) ПО. Сами этапы включают в себя:

- подготовку организации (разработчиков);
- защиту ПО (защиту всех компонентов ПО от несанкционированного доступа);
- производство хорошо защищённого ПО (минимальное количество уязвимостей в релизах ПО);
- реагирование на уязвимости (выявление остаточных уязвимостей в выпускаемом ПО и соответствующее реагирование на них);

Условно, самый простой цикл с точки зрения акцента именно на разработку ПО включает в себя 5 шагов:

- сбор и анализ требований безопасности к разрабатываемому ПО;
- проектирование (дизайн) ПО;
- реализация ПО (непосредственно разработка ПО);
- тестирование ПО;
- деплой ПО (выпуск, публикация или т.п.).

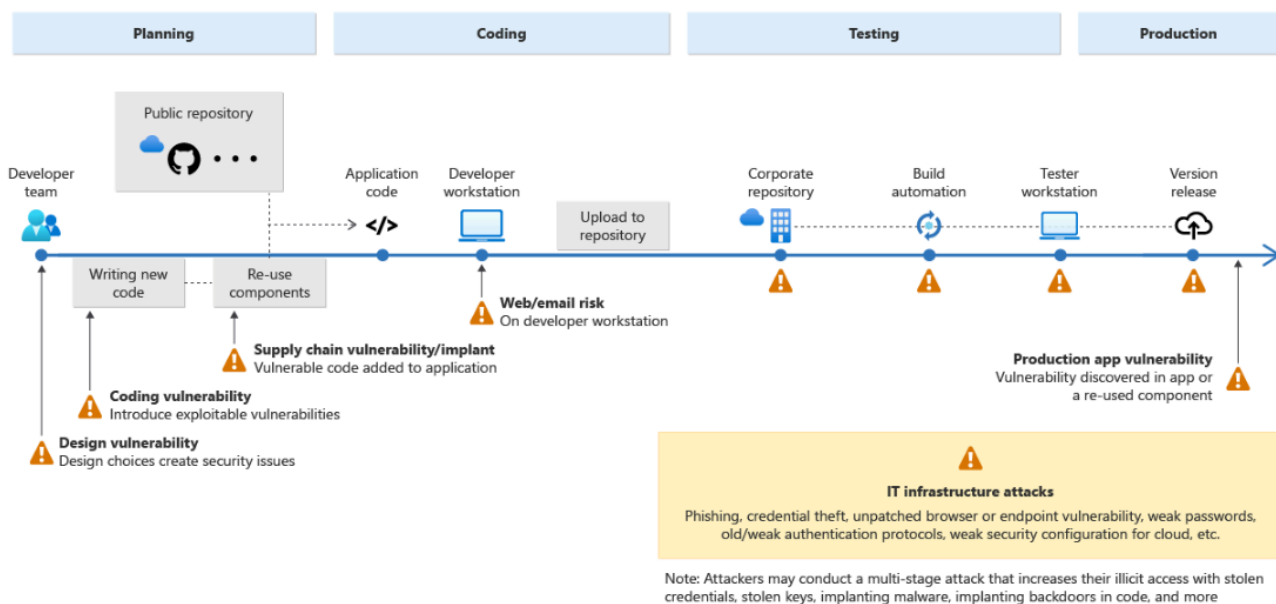


Принципы безопасной разработки приложений

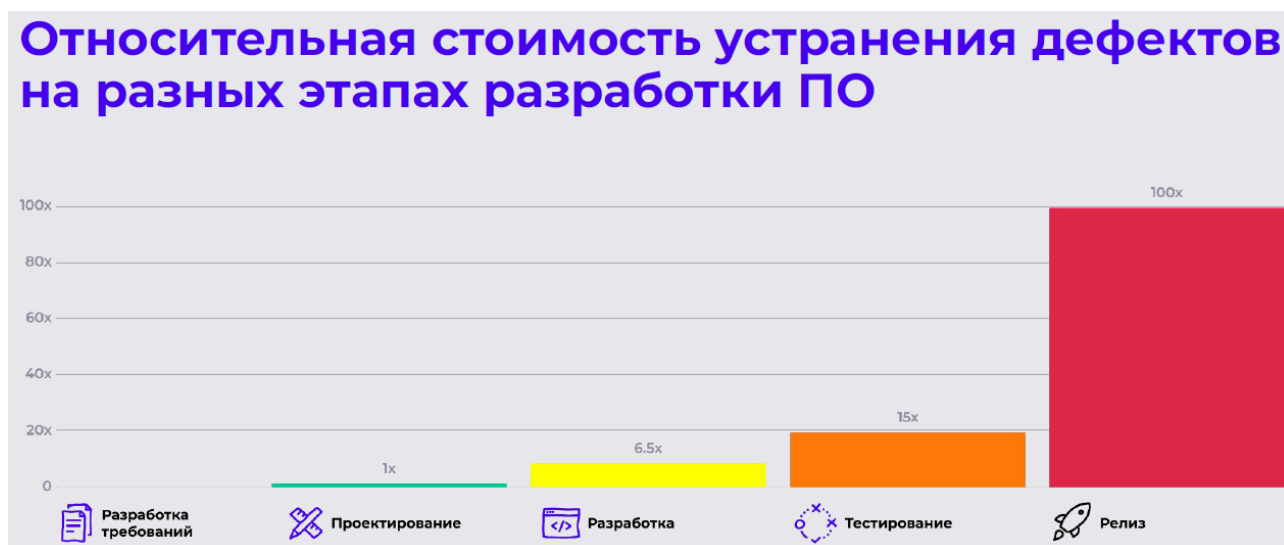
Согласно ISO/IEC 27034-1 [3] выделяются следующие ключевые принципы безопасной разработки приложений:

- безопасность является требованием: требования безопасности приложений должны трактоваться таким же образом, как и требования к функциональным возможностям, качеству и удобству в эксплуатации (юзабилити);
- безопасность приложений зависит от контекста, включая бизнес-контекст (сферу бизнеса организации), регуляторный контекст, технологический контекст;
- выделение соответствующих инвестиций в обеспечение безопасности приложений (разумная достаточность);
- безопасность приложений должна демонстрироваться (приложение не может быть объявлено безопасным, если условно «аудитор» не подтвердил это);

🔥 Дополнительно стоит отметить принцип Shift-Left Security, т. е. внедрение практик и мер обеспечения безопасности приложения, а также других мер защиты в процесс разработки ПО, т.е. SDLC, так рано, как только это возможно.



Принцип Shift-Left Security обусловлен тем, что относительная стоимость (время, трудозатраты, прямые финансовые затраты или т. п.) устранения уязвимостей или учёта требований по безопасности резко возрастает при движении вправо по шкале SDLC [5]:

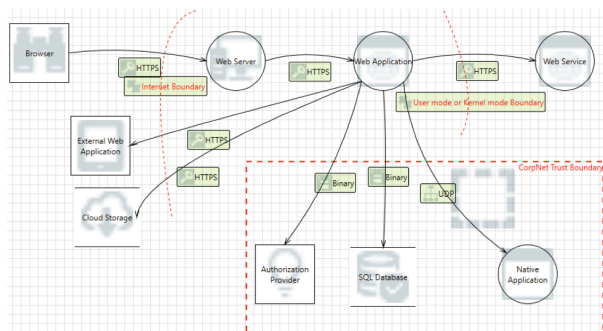


Отличительная особенность моделирования угроз при разработке ПО

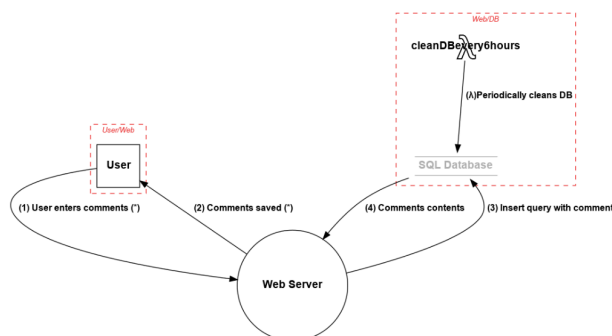
Моделирование угроз для разрабатываемых приложений является неотъемлемой частью SDL. Отличительной особенностью моделирования угроз при разработке ПО

от классического процесса определения актуальных угроз является акцент на визуализацию – построение диаграмм.

Для этого могут использоваться различные программные утилиты, например, от Корпорации Microsoft или сообщества Open Web Application Security Project (OWASP), или обычный Microsoft Visio:



[Microsoft Threat Modeling Tool](#)



[OWASP Threat Dragon](#)

Популярные best practices и framework

ISO/IEC 27034 Application Security и ГОСТ

Начнём с уже знакомой нам серии стандартов ISO/IEC 27k, в рамках неё есть отдельная ветка стандартов – ISO/IEC 27034 Application Security (безопасность приложений), включающая:

- обзор и общие понятия;
- нормативную структуру организации;
- процесс менеджмента безопасности приложений;
- структуры данных протоколов и мер обеспечения безопасности приложений;
- практические примеры;
- основы прогнозирования доверия.

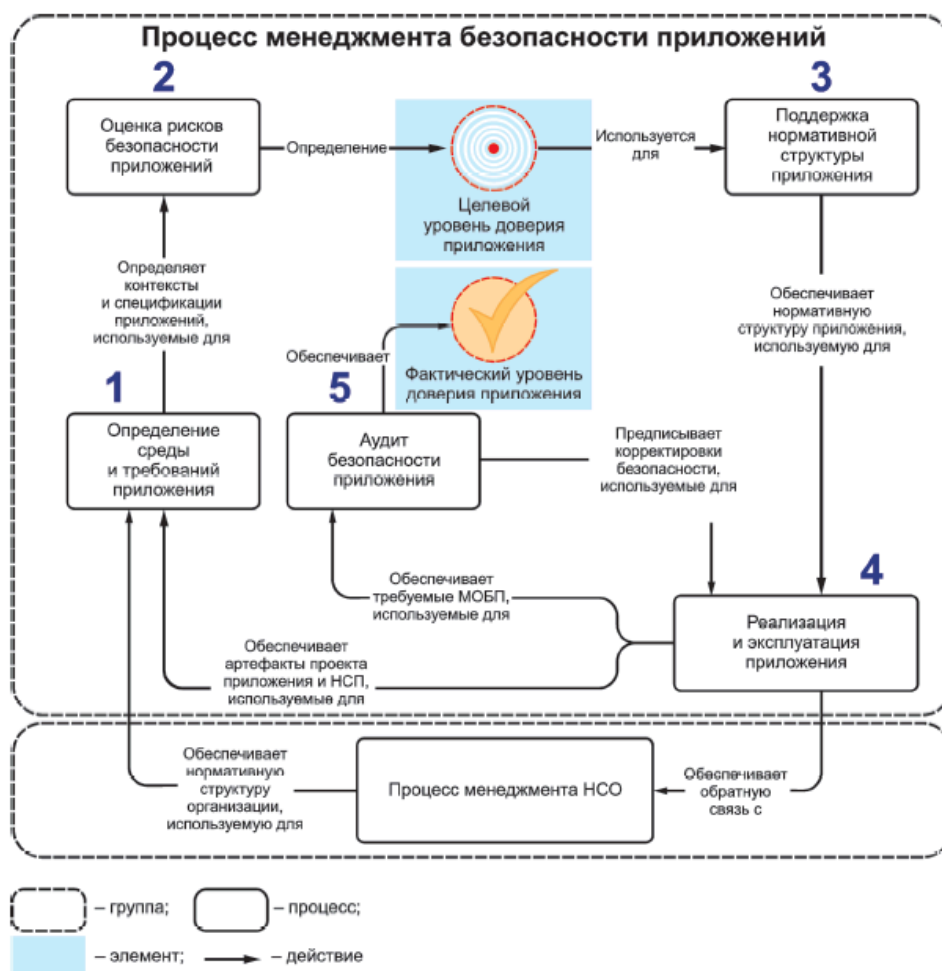
В Российской Федерации значительная часть этих стандартов (редакций стандартов) получала переиздание в формате государственных стандартов – ГОСТ Р ИСО/МЭК.

Согласно ISO/IEC 27034-3 [3] процесс менеджмента безопасности приложения включает в себя 5 этапов:

- определение среды и требований приложения
- оценка рисков, связанных с безопасностью приложения
- создание и поддержка нормативной структуры приложения
- подготовка к работе и эксплуатацию приложения
- аудит безопасности приложения

Первые три этапа данного процесса направлены на определение и подтверждение соответствующих мер обеспечения безопасности приложения для конкретного приложения. Учитывая, что безопасность на начальном этапе является основополагающим фактором безопасности приложения, оптимальной точкой для определения требований безопасности для проекта в области разработки ПО является этап начального планирования. Определение требований безопасности на начальном этапе помогает проектным группам определять ключевые этапы и результаты, а также позволяет интегрировать безопасность таким образом, чтобы свести к минимуму любые нарушения планов и графиков.

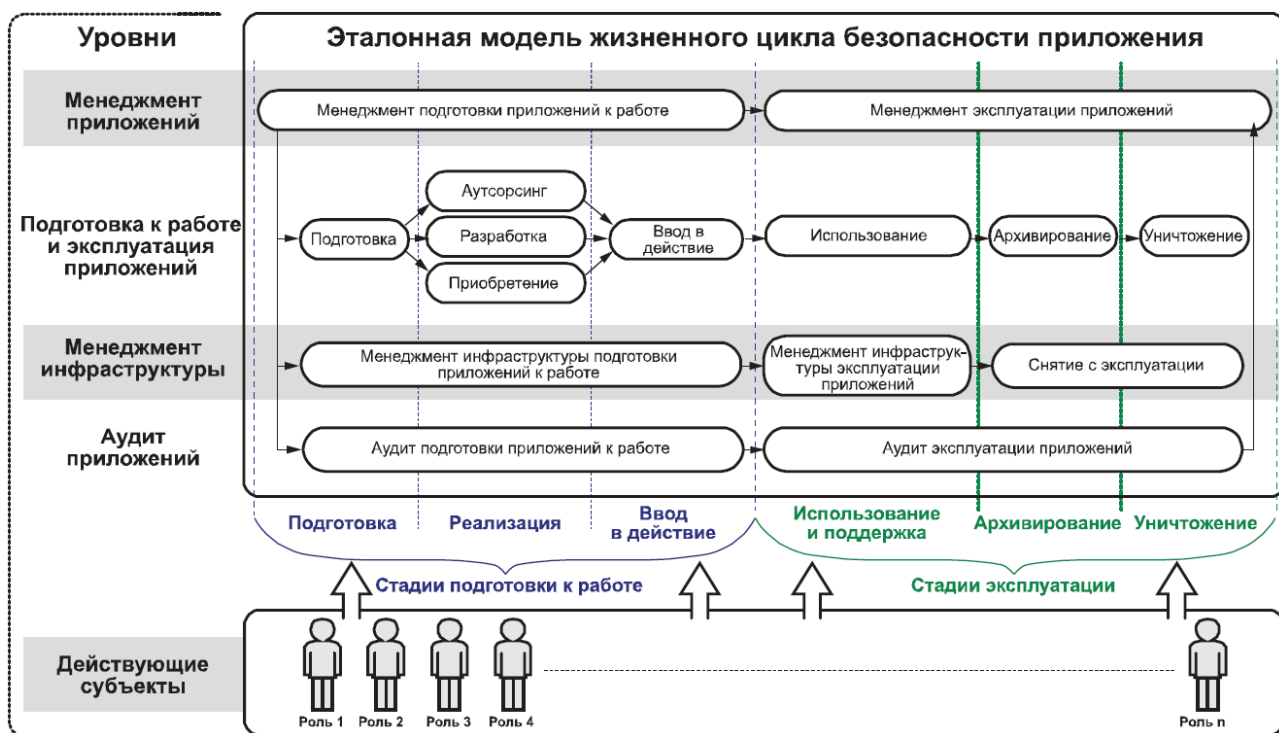
Заключительные два этапа процесса направлены на внедрение и проверку соответствующих мер обеспечения безопасности приложения.



Здесь же стоит отметить жизненный цикл приложения, в котором выделяют 2 основные стадии:

- создание приложения:
 - подготовка;
 - реализация;
 - ввод в действие.
- эксплуатация приложения
 - использования;
 - поддержки;
 - архивирование;
 - уничтожение.

В каждой фазе выполняются мероприятия, связанные с подготовкой к работе, эксплуатацией и аудитом приложений, а также менеджментом инфраструктуры и менеджментом приложений, что требует участия различных специалистов, а не только команды разработчиков.



Стоит отметить и разобрать понятие нормативной структуры организации, под которой понимается общая организационная структура, содержащая подмножество процессов и компонентов организации (политик, методов, ролей), а также инструментальных средств, которые участвуют в обеспечении безопасности приложений и являются стандартными внутри организации.

Цели внедрения нормативной структуры организации состоят в следующем:

- назначение лиц, ответственных за обеспечение безопасности приложений и установку процесса, который содействует повышению безопасности приложений;
- создание условий для того, чтобы ответственные лица, принимающие решения, могли одобрять все элементы (компоненты, роли и процессы), связанные с безопасностью приложений, а участники и заинтересованные стороны могли принять эти условия;
- минимизация сопротивления изменениям, вызванным внедрением новых элементов безопасности приложений;
- стандартизация элементов безопасности приложений с целью обеспечения единообразия их внедрения и верификации во всей организации;
- повышение уровня зрелости организации;
- создание механизмов, позволяющих более экономно внедрять меры безопасности (например, с помощью повторного использования существующих утвержденных элементов безопасности приложений).

Дополнительно стоит здесь упомянуть ГОСТ Р 56939-2016 [4], который устанавливает общие требования к содержанию и порядку выполнения работ, связанных с созданием безопасного (защищённого) ПО и формированием (поддержанием) среды обеспечения и оперативного устранения выявленных пользователями ошибок ПО и уязвимостей.

В рамках данного стандарта определены следующие группы мер по разработке безопасного ПО:

- меры, реализуемые при выполнении анализа требований к ПО;
- меры, реализуемые при выполнении проектирования архитектуры программы;
- меры, реализуемые при выполнении конструирования и комплексирования ПО;
- меры, реализуемые при выполнении квалификационного тестирования ПО;
- меры, реализуемые при выполнении инсталляции программы и поддержки приёмки ПО;
- меры, реализуемые при решении проблем в ПО в процессе эксплуатации;
- меры, реализуемые в процессе менеджмента документацией и конфигурацией программы;
- меры, реализуемые в процессе менеджмента инфраструктурой среды разработки ПО;
- меры, реализуемые в процессе менеджмента людскими ресурсами.

 Таким образом, обеспечение безопасности приложений не сводится только к прикладной стороне вопроса, связанной непосредственно с написанием программного кода, и не ограничивается только командой разработчиков.

NIST SSDF

Secure Software Development Framework (SSDF) [2], включает в себя практики, объединённые в 4 группы:

Подготовка организации (разработчиков):

- определить требования к безопасности ПО;
- определить роли и зоны ответственности;
- внедрить вспомогательные инструменты (Supporting Toolchains);

- определить и использовать критерии для проверок безопасности ПО;
- внедрить и поддерживать безопасную среду для разработки ПО.

Защита ПО:

- защитить все формы кода от несанкционированного доступа;
- предоставить механизмы проверки целостности выпуска ПО;
- архивировать и защищать каждую версию ПО.

Производство хорошо защищённого ПО:

- разрабатывать ПО, отвечающее требованиям безопасности и снижающее риски безопасности;
- проверять (review) дизайн, чтобы убедиться соответствии требованиям безопасности;
- убеждаться (verify), что стороннее ПО соответствует требованиям безопасности;
- повторно использовать существующее, хорошо защищённое ПО, вместо дублирования функций (если это возможно);
- создавать программный код, придерживаясь методов (стандартов) безопасного кодирования;
- настраивать процессы компиляции, интерпретатора и сборки для повышения безопасности исполняемых файлов;
- проверять (review) и/или анализировать код для выявления уязвимостей и проверки соответствия требованиям безопасности (verify);
- тестировать (test) исполняемого кода для выявления уязвимостей и проверки соответствия требованиям безопасности (verify);
- настраивать ПО для использования безопасных настроек по умолчанию.

Реагирование на уязвимости (выявление остаточных уязвимостей в выпускаемом ПО и соответствующее реагировать на них):

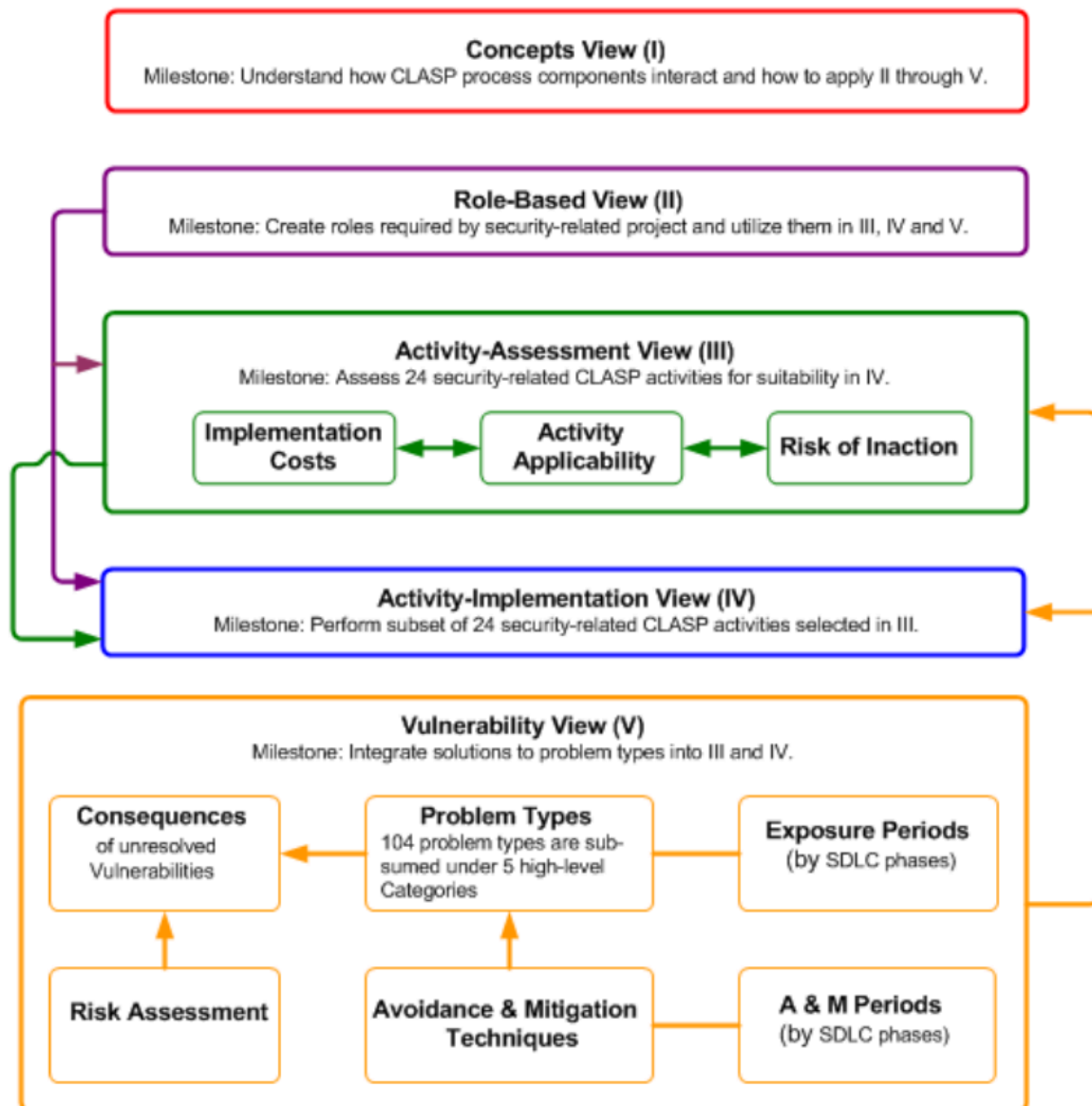
- постоянно выявлять уязвимости в ПО и подтверждать их;
- оценивать, приоритизировать и устранять уязвимости;
- анализировать уязвимости для выявления их коренных причин (root causes analysis).

OWASP CLASP и SSDF

На сегодняшний день сообщество OWASP – один из лидеров по количеству рекомендаций и материалов по линии безопасной разработки ПО.

Рассмотрение их материалов стоит начать, с уже практически не применяемого, но достойного упоминания Comprehensive, Lightweight Application Security Process (CLASP), который затрагивал 5 аспектов (CLASP Views):

- концепции (Concepts View);
- роли (Role-Based View);
- оценка (Activity-Assessment View);
- реализация (Activity-Implementation View);
- уязвимости (Vulnerability View).



Сейчас же OWASP в рамках [Secure Coding Practices](#) предлагает свой SSDF, делая акцент:

- на чётком определении ролей и зон ответственности;

- в обеспечении команды разработчиков необходимым обучением (тренингами);
- во внедрении SDL;
- в установлении стандартов безопасного кодирования (secure coding standards), предлагается [OWASP Developer Guide](#);
- на создании повторно используемой библиотеки объектов, предлагается [OWASP Enterprise Security API \(ESAPI\)](#);
- на верификации эффективности применяемых мер, предлагается [OWASP Application Security Verification Standard \(ASVS\)](#);
- на установлении безопасных методов аутсорсинга разработки ПО.



Большая часть материалов доступна в актуальном состоянии в официальном [GitHub-репозитории OWASP](#). Обязательно познакомьтесь с ними.

Приведённые выше материалы, лишний раз подчёркивают необходимость расширения области безопасной разработки приложений с этапа написания кода до организационных и смежных технических мероприятий.

Рекомендации по безопасному написанию кода

Непосредственно рекомендации по безопасному написанию кода в данном разделе базируются на Secure Coding Practices от OWASP, которые для удобства были представлены в формате [Secure Coding Practices Checklist](#). Данные рекомендации не зависят от технологического стека, нацелены на устранение наиболее распространённых уязвимостей в ПО, но с акцентом на веб-приложения.

Валидация входных данных (Input validation)

- проверка всех вводимых данных на стороне доверенной системы (на стороне сервера, а не клиента);
- определение всех источников данных и их классификация на доверенные и недоверенные;

- валидация всех данных, полученных из недоверенных источников (базы данных, файловые потоки и т.д.);
- централизованная процедура валидации для всего приложения;
- задание кодировки (например, UTF-8) для всех источников входных данных (canonicalization);
- кодирование входных данных в общий набор символов перед валидацией;
- отклонение ввода в случае всех ошибок валидации;
- проверка после завершения декодирования UTF-8, если система поддерживает расширенные наборы символов UTF-8;
- валидация всех данных, предоставляемых клиентом, перед обработкой;
- верификация содержания только ASCII-символов в значениях заголовков протоколов в запросах и ответах;
- валидация данных, полученных при перенаправлении;
- валидация на соответствие ожидаемым типам данных с использованием списка «разрешённых», а не списка «запрещённых»;
- валидация диапазона данных;
- валидация длины данных;
- применять дополнительные средства контроля, если необходимо разрешать ввод потенциально опасных данных;
- применять дополнительные дискретные проверки, если стандартная процедура валидации не может справиться с некоторыми входными данными;
- применять канонизацию (canonicalization) для борьбы с обфускацией (запутыванием данных);

Кодирование выходных данных (Output encoding)

- кодирование выходных данных на стороне доверенной системы (на стороне сервера, а не клиента);
- применять стандартную, проверенную процедуру для каждого типа исходящего кодирования;
- указывать кодировку (например, UTF-8) для всех выводимых данных;
- контекстное кодирование всех данных, возвращаемых клиенту из ненадёжных источников;
- убедиться, что кодировка выходных данных безопасна для всех целевых систем;
- контекстная санитизация (sanitize) всех выходных данных от недоверенных данных (входящих запросов) в исходящие запросы к SQL, XML и LDAP (т.е.

удаление / экранирование неправильных или небезопасных символов в данных);

- санитизация всех выходных данных от недоверенных данных в командах к операционной системе;

Аутентификация и управление паролями (Authentication and password management)

- требовать аутентификацию для всех страниц и ресурсов, кроме тех, которые специально предназначены для общего доступа;
- все средства контроля аутентификации должны применяться на доверенной системе;
- создавать и использовать стандартные, проверенные службы аутентификации, когда это возможно;
- централизованная реализация для всех элементов управления аутентификацией, включая библиотеки, вызывающие внешние сервисы аутентификации;
- отделять логику аутентификации от запрашиваемого ресурса и использовать перенаправление к централизованному сервису аутентификации и обратно;
- все средства управления аутентификацией должны работать в безопасном режиме;
- все функции администрирования и управления учётными записями должны быть, по крайней мере, столь же безопасными, как и основной механизм аутентификации;
- применять криптографически стойкие односторонние хеши с «солью», если приложение управляет хранилищем учётных данных;
- хешировать пароли на стороне доверенной системы (на стороне сервера, а не клиента);
- проверять данные аутентификации, только после завершения ввода всех данных;
- в ответах на отказ аутентификации не указывать, какая часть аутентификационных данных была неверной;
- применять аутентификацию для подключения к внешним системам, содержащим конфиденциальную информацию или функции;
- хранить в защищённом хранилище аутентификационные данные для доступа к внешним по отношению к приложению сервисам;
- применять для передачи аутентификационных данных только HTTP POST-запросы;

- передавать не временные пароли только по зашифрованному соединению или в виде зашифрованных данных;
- соблюдать требования к сложности паролей;
- соблюдать требования к длине пароля;
- скрывать вводимые значения пароля на экране пользователя;
- отключать учётную запись после установленного количества неверных попыток входа в систему;
- операции по сбросу и изменению пароля требуют такого же уровня контроля, как создание учётной записи и аутентификация;
- вопросы для сброса пароля должны поддерживать достаточно случайные ответы (Секретный вопрос типа "девичьей фамилии" и т.п.);
- при использовании сброса пароля по электронной почте отправлять его только на заранее зарегистрированный адрес с временной ссылкой/паролем;
- временные пароли и ссылки должны иметь короткий срок действия;
- обеспечить смену временных паролей при следующем использовании;
- уведомлять пользователей о сбросе пароля;
- предотвращать повторное использование паролей;
- срок действия паролей должен составлять не менее одного дня, чтобы предотвращать атаки на повторное использование;
- обеспечить принудительную смену паролей в соответствии с требованиями, при этом время между сменой паролей должно контролироваться административно;
- отключить функцию «запомнить меня» для полей паролей;
- о последнем использовании (успешном или неуспешном) учётной записи пользователя сообщать пользователю при его следующем успешном входе в систему;
- реализовать мониторинг для выявления атак на несколько учётных записей пользователей, использующих один и тот же пароль (password spraying);
- изменить все пароли и идентификаторы пользователей, предоставляемые поставщиком по умолчанию, или отключить соответствующие учётные записи;
- повторно аутентифицировать пользователей перед выполнением критических операций;
- применять многофакторную аутентификацию для особо важных и дорогостоящих учётных записей;
- если для аутентификации используется код сторонних разработчиков, проверить его на отсутствие вредоносного кода;

Управление сессиями (Session management)

- Применяя средства управления сеансами сервера, приложение должно распознавать только эти идентификаторы сеансов как действительные.
- Создание идентификатора сеанса всегда должно осуществляться на стороне доверенной системе (на стороне сервера, а не клиента.)
- Средства управления сеансами должны использовать хорошо проверенные алгоритмы, обеспечивающие достаточно случайные идентификаторы сеансов.
- Установить путь для cookies, содержащих аутентифицированные идентификаторы сеансов, на значение, соответствующее ограничениям сайта.
- Функции выхода из системы должны полностью завершать связанную сессию или соединение.
- Функции выхода из системы должны быть доступны на всех страницах, защищённых авторизацией.
- Установить как можно более короткое время бездействия сеанса, исходя из соотношения рисков и функциональных требований бизнеса.
- Запретить постоянные входы в систему и обеспечить периодическое завершение сеанса, даже если он активен.
- Если сеанс был создан до входа в систему, то после успешного входа в систему закрыть этот сеанс и создать новый.
- Генерировать новый идентификатор сессии при любой повторной аутентификации.
- Не допускать одновременного входа в систему с одним и тем же идентификатором пользователя.
- Не раскрывать идентификаторы сеансов в URL, сообщениях об ошибках и журналах.
- Реализовать соответствующие средства контроля доступа для защиты данных сеанса на стороне сервера от несанкционированного доступа со стороны других пользователей сервера.
- Генерировать новый идентификатор сессии и периодически деактивировать старый.
- Генерировать новый идентификатор сеанса при изменении безопасности соединения с HTTP на HTTPS, что может произойти при аутентификации.
- Последовательно использовать HTTPS, а не переключаться между HTTP и HTTPS.

- Дополнить стандартное управление сеансами для чувствительных операций на стороне сервера, таких как управление учётными записями, используя для каждой сессии надёжные случайные маркеры или параметры.
- Дополнить стандартное управление сеансами для очень чувствительных или критических операций, используя не сеансовые, а запросовые маркеры или параметры.
- Установить атрибут «безопасный» для файлов cookie, передаваемых по TLS-соединению.
- Устанавливать cookie-файлы с атрибутом HttpOnly, если только вы специально не требуете, чтобы клиентские сценарии в вашем приложении считывали или устанавливали значение cookie-файла.

Управление доступом (Access control)

- Использовать для принятия решений об авторизации доступа только доверенные системные объекты, например, сеансовые объекты на стороне сервера.
- Использовать для авторизации доступа единый компонент в масштабе всего сайта, это касается и библиотек, вызывающих внешние службы авторизации.
- Контроль доступа должен быть безопасным.
- Отказывать в доступе, если приложение не может получить информацию о конфигурации безопасности.
- Применять средства контроля авторизации при каждом запросе, включая запросы, выполняемые скриптами на стороне сервера.
- Отделять привилегированную логику от остального кода приложения.
- Ограничивать доступ к файлам и другим ресурсам, в т.ч. находящимся вне прямого контроля приложения – только для авторизованных пользователей.
- Ограничивать доступ к защищённым URL-адреса – только для авторизованных пользователей.
- Ограничивать доступ к защищённым функциям – только для авторизованных пользователей.
- Ограничивать доступ к прямым ссылкам на объекты – только для авторизованных пользователей.
- Ограничивать доступ к сервисам – только для авторизованных пользователей.
- Ограничивать доступ к данным приложения – только для авторизованных пользователей.

- Ограничивать доступ к атрибутам пользователей и данных, а также к информации о политике, используемой средствами управления доступом.
- Ограничивать доступ к конфигурационной информации, имеющей отношение к безопасности – только для авторизованных пользователей.
- Реализация на стороне сервера и представление правил управления доступом на уровне представлений (Layer 6 модели OSI) должны совпадать.
- Применять шифрование и проверку целостности на стороне сервера, если данные о состоянии (state data) должны храниться на клиенте.
- Обеспечивать соответствие логических потоков приложения бизнес-правилам.
- Ограничивать количество транзакций, которые может выполнить один пользователь или устройство за определённое время, достаточно низкое для предотвращения автоматизированных атак, но превышающее реальные бизнес-требования.
- Использовать заголовок «referer» только в качестве дополнительной проверки, он никогда не должен быть единственной проверкой авторизации, т.к. может быть подделан.
- Периодически перепроверять авторизацию пользователя, чтобы убедиться, что его привилегии не изменились, а если изменились, то вывести из системы и заставить пользователя пройти повторную аутентификацию, если разрешены длительные сеансы аутентификации.
- Реализовать аудит учётных записей и принудительное отключение неиспользуемых учётных записей.
- Приложение должно поддерживать отключение учётных записей и завершение сеансов при прекращении авторизации.
- Сервисные учётные записи или учётные записи, поддерживающие соединения с внешними системами или из них, должны иметь минимально возможные привилегии.
- Создание политики управления доступом для документирования бизнес-правил приложения, типов данных и критериев и/или процессов авторизации доступа, чтобы обеспечить надлежащее предоставление и контроль доступа (это включает определение требований доступа как к данным, так и к системным ресурсам).

Криптографическая защита (Cryptographic practices)

- Все криптографические функции, используемые для защиты секретов от пользователя приложения, должны быть реализованы на стороне доверенной системы;
- Защита секретов от несанкционированного доступа;
- Криптографические модули должны выходить из строя безопасно;
- Все случайные числа, случайные имена файлов, случайные GUID и случайные строки должны генерироваться с помощью утверждённого генератора случайных чисел криптографического модуля;
- Криптографические модули, используемые приложением, должны соответствовать заданному стандарту (FIPS 140-2 или НМД ФСБ России);
- Разработать и применять политику и процесс управления криптографическими ключами;

Обработка ошибок и регистрация событий (Error handling and logging)

- Не раскрывать конфиденциальную информацию в ответах на ошибки, включая системные данные, идентификаторы сеансов и/или информацию об учётных записях.
- Применять обработчики ошибок, не отображающие отладочную информацию или stack trace.
- Реализовывать общие сообщения об ошибках и использовать пользовательские страницы ошибок.
- Приложение должно обрабатывать ошибки приложения, а не полагаться на конфигурацию сервера.
- Правильно освобождать выделенную память при возникновении ошибок.
- Логика обработки ошибок, связанная с элементами управления безопасностью, должна запрещать доступ по умолчанию.
- Все элементы управления регистрацией событий (журналированием) должны быть реализованы на стороне доверенной системы.
- Средства ведения журналов должны поддерживать как успешные, так и неуспешные события безопасности.
- Убедиться, что журналы содержат важные (нужные) данные о событиях безопасности.

- Убедиться, что записи журнала (события безопасности), содержащие недостоверные данные, не будут выполняться как код, в предназначенном для просмотра журнала интерфейсе или ПО.
- Ограничить доступ к журналам только для уполномоченных лиц.
- Использовать централизованную процедуру для всех операций по ведению журналов.
- Не хранить в журналах конфиденциальную информацию, включая ненужные системные данные, идентификаторы сеансов или пароли.
- Обеспечить наличие механизма для проведения анализа журналов.
- Регистрировать все ошибки проверки ввода.
- Регистрировать все попытки аутентификации, особенно неудачи.
- Регистрировать все сбои в управлении доступом.
- Регистрировать все очевидные события несанкционированного доступа, включая неожиданные изменения данных состояния (state data).
- Регистрировать попытки подключения с недействительными или просроченными токенами сеансов.
- Регистрировать все системные исключения.
- Регистрировать все административные функции, включая изменения параметров конфигурации безопасности.
- Регистрировать все сбои TLS-соединения с внутренним сервером.
- Регистрировать отказы криптографических модулей.
- Применять хеш-функции для проверки целостности записей журналов.

Защита данных (Data protection)

- Реализовать принцип наименьших привилегий, ограничивая доступ пользователей только к тем функциям, данным и системной информации, которые необходимы для выполнения их задач.
- Защищать все кэшированные или временные копии конфиденциальных данных, хранящиеся на сервере, от несанкционированного доступа и очищать эти временные рабочие файлы, как только в них отпадает необходимость.
- Шифровать высокочувствительную хранимую информацию, например, данные проверки подлинности, даже если она находится на стороне сервера
- Защищать исходный код сервера от загрузки (download) пользователем.
- Не хранить пароли, строки подключения и другую конфиденциальную информацию открытым текстом или любым некриптографически защищённым способом на стороне клиента.

- Удалять комментарии в доступном пользователю производственном коде, которые могут раскрыть внутреннюю систему или другую конфиденциальную информацию.
- Удалять ненужную документацию по приложениям и системам (из открытого доступа), т.к. она может раскрыть полезную информацию злоумышленникам.
- Не включать конфиденциальную информацию в параметры HTTP GET-запроса.
- Отключать функции автоматического заполнения форм, которые могут содержать конфиденциальную информацию, включая аутентификационную.
- Отключать кэширование на стороне клиента на страницах, содержащих конфиденциальную информацию.
- Приложение должно поддерживать удаление конфиденциальных данных, когда они больше не нужны.
- Реализовывать соответствующие средства контроля доступа к конфиденциальным данным, хранящимся на сервере (кэшированные данные, временные файлы и данные, доступ к которым должен быть только у определённых пользователей системы).

Защита коммуникаций (Communication security)

- Применять шифрование для передачи всей конфиденциальной информации, включая TLS для защиты соединения и может быть дополнено дискретным шифрованием конфиденциальных файлов или соединений, не основанных на HTTP.
- TLS-сертификаты должны быть действительными, иметь правильное доменное имя, не быть просроченными и устанавливаться с промежуточными сертификатами, если это необходимо.
- Неудачные TLS-соединения не должны возвращаться к небезопасным соединениям.
- Применять TLS-соединения для всего содержимого, требующего аутентифицированного доступа, и для всей другой конфиденциальной информации.
- Применять TLS для соединений с внешними системами, которые содержат конфиденциальную информацию или функции.
- Применять единую стандартную реализацию TLS, настроенную соответствующим образом.
- Указывать кодировки символов для всех соединений.

- Фильтровать параметры, содержащие конфиденциальную информацию, в HTTP-ссылке при переходе на внешние сайты.

Безопасная конфигурация системы (System configuration)

- Убедиться, что системные компоненты и ПО используют последнюю версию.
- Убедиться, что системные компоненты и ПО имеют все исправления, выпущенные для используемой версии.
- Отключить листинги каталогов.
- Ограничить учётные записи веб-серверов, процессов и служб минимально возможными привилегиями.
- При возникновении исключений выполнять безопасное завершение работы.
- Удалить всю ненужную функциональность и файлы.
- Перед развёртыванием удалить тестовый код или любую функциональность, не предназначенную для использования в продуктиве (production).
- Предотвратить раскрытие структуры каталогов в файле robots.txt, поместив каталоги, не предназначенные для публичного индексирования, в изолированный родительский каталог.
- Определить, какие HTTP-методы, GET или POST, будет поддерживать приложение, и будут ли они по-разному обрабатываться на разных страницах приложения.
- Отключить ненужные HTTP-методы.
- Убедиться, что различные версии HTTP настроены одинаково, если веб-сервер работает с различными версиями HTTP.
- Удалите из заголовков HTTP-ответов ненужную информацию, связанную с ОС, версией веб-сервера и фреймворками приложений.
- Хранилище конфигурации безопасности для приложения должно иметь возможность вывода в человекочитаемом виде для проведения аудита.
- Внедрить систему управления активами и зарегистрировать в ней системные компоненты и ПО.
- Изолировать среду разработки от продуктивной среды и предоставлять доступ только для авторизованных группам разработчиков и тестировщиков.
- Внедрить систему контроля изменений в ПО для управления и регистрации изменений в коде как в процессе разработки, так и в процессе эксплуатации.

Безопасность баз данных (Database security)

- Использование сильно типизированных, параметризованных запросов.
- Использовать проверку ввода и кодировку вывода и обязательно учитывать метасимволы, если они не работают, то не выполнять запрос.
- Убедиться, что переменные сильно типизированы.
- При обращении к базе данных приложение должно использовать минимально возможный уровень привилегий.
- Применять защищённые учётные данные для доступа к базе данных.
- Строки соединений не должны быть жестко закодированы в приложении, строки соединений должны храниться в отдельном конфигурационном файле на стороне доверенной системы и должны быть зашифрованы.
- Применять хранимые процедуры, для абстрагирования доступа к данным и снятия прав доступа к базовым таблицам базы данных.
- Закрывать соединение как можно быстрее.
- Удалять или изменять все административные пароли баз данных по умолчанию.
- Отключить все ненужные функции базы данных.
- Удалить ненужный контент поставщика по умолчанию (например, образцы схем).
- Отключить все учётные записи по умолчанию, которые не требуются для поддержки бизнес-требований.
- Приложение должно подключаться к базе данных с различными учётными данными для каждого разграничения доверия (например, пользователь, пользователь только для чтения, гость, администраторы).

Безопасность файлов (File management)

- Не передавать предоставленные пользователем данные непосредственно в любую функцию как динамическое включение.
- Ограничить тип загружаемых (upload) файлов только теми типами, которые необходимы для рабочих целей.
- Проверять соответствие загружаемых (upload) файлов ожидаемому типу путем проверки заголовков файлов, а не по расширению файла.
- Не сохранять файлы в том же веб-контексте, что и приложение.
- Запретить или ограничить загрузку любых файлов, которые могут быть интерпретированы веб-сервером.
- Отключить привилегии выполнения файлов в каталогах загрузки.

- Реализовать безопасную загрузку (upload) в UNIX путём монтирования каталога целевого файла как логического диска с использованием связанного пути или chrooted-окружения.
- При обращении к существующим файлам, использовать список разрешённых имен и типов файлов.
- Не передавать пользовательские данные в динамическое перенаправление
- Не передавать пути к каталогам или файлам, использовать индексные значения, сопоставленные с заранее определённым списком путей.
- Не передавать клиенту абсолютный путь к файлу.
- Убедиться, что файлы и ресурсы приложения доступны только для чтения.
- Проверять загружаемые (upload) пользователем файлы на наличие вирусов и вредоносного ПО.

Безопасность данных в памяти (Memory management)

- Использовать средства контроля ввода и вывода недоверенных данных.
- Убедиться, что буфер имеет заданный размер.
- Убедиться, что корректно обрабатывается завершение NULL, если используются функции, принимающие на вход число байт.
- Проверить границы буфера, если функция вызывается в цикле, и защитите её от переполнения.
- Усекать все входные строки до разумной длины перед передачей их другим функциям.
- Специально закрывать ресурсы, не полагайтесь на «сборку мусора».
- Использовать неисполняемые стеки, когда это возможно.
- Избегать использования известных уязвимых функций.
- Правильно освобождать выделенную память по завершении работы функций и во всех точках выхода.
- Перезаписывать любую конфиденциальную информацию, хранящуюся в выделенной памяти, во всех точках выхода из функции.

Общие практики кодирования (General coding practices)

- Использовать протестированный и одобренный код вместо создания нового кода для решения общих задач.

- Использовать встроенные API для выполнения задач в ОС, не позволять приложению выдавать команды непосредственно в ОС, особенно с помощью командных оболочек, иницируемых данным приложением.
- Использовать контрольные суммы или хеши для проверки целостности интерпретируемого кода, библиотек, исполняемых и конфигурационных файлов.
- Использовать блокировку для предотвращения нескольких одновременных запросов или механизм синхронизации для предотвращения условий «гонки».
- Защищать общие переменные и ресурсы от нежелательного одновременного доступа.
- Явно инициализировать все переменные и другие хранилища данных либо при их объявлении, либо непосредственно перед первым использованием.
- Повышать привилегии как можно позже и снижать их как можно раньше, в случаях, когда приложение должно работать с повышенными привилегиями.
- Избегать ошибки при вычислениях, понимая, что лежит в основе представления данных в языке программирования.
- Не передавать пользовательские данные в функцию как динамическое включение.
- Запретить пользователям генерировать новый или изменять существующий код.
- Проанализировать все вторичные приложения, код сторонних разработчиков и библиотеки, чтобы определить необходимость в них и убедиться в их безопасной функциональности.
- Реализовать безопасное обновление с использованием зашифрованных (защищённых) каналов.

Основные способы верификации требований к безопасности для приложений

К основным способам верификации требований безопасности приложений можно отнести:

- анализ (инспекция) кода (code review);
- статический анализ (static application security testing (SAST));
- динамический анализ (dynamic application security testing (DAST));
- компонентный анализ (software composition analysis (SCA));

- анализ внешних зависимостей (open source analysis (OSA));
- анализ мобильных приложений (mobile application security testing (MAST));
- поиск секретов в исходном коде (detect exposed secrets in code);
- фаззинг-тестирование ПО (fuzz testing);
- тестирование на проникновение (penetration testing);

в данном случае не рассматриваем функциональное тестирование, тестирование производительности и другие тесты, явно не относящиеся к безопасности.

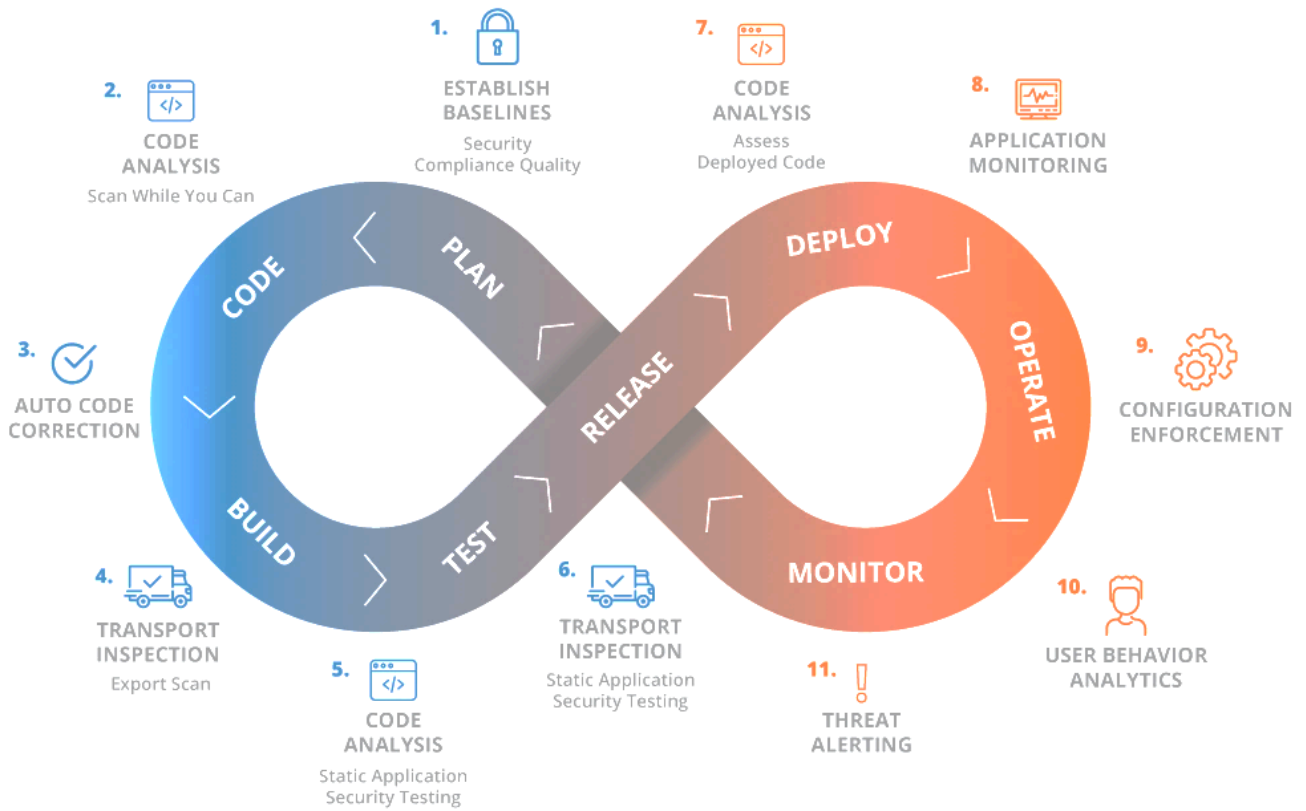
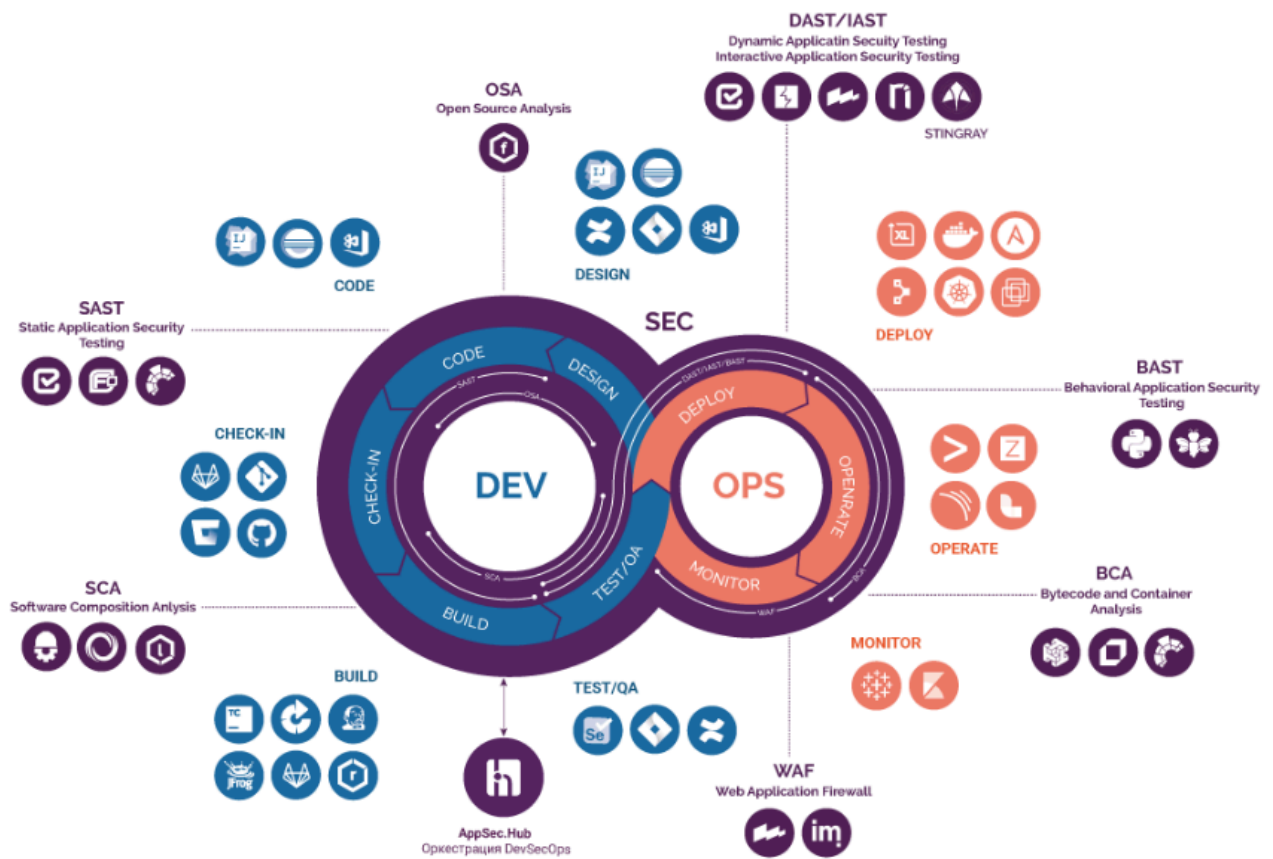
[OWASP ASVS](#) предлагает следующее распределение способов верификации:

	Applicability	Building			Building, Configuration, Deployment Assurance and Verification			Assurance and Verification	
Level 1	All apps		Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Penetration Testing	DAST
Level 2	All apps	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Level 3	High Assurance	Security Architecture and Reviews	Secure Coding	Standards and checklists	Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Legend		Acceptable	Suitable						

Безопасность в цикле CI/CD – DevSecOps

Рассмотрев рекомендации по безопасному написанию кода и способы верификации, стоит рассмотреть, как всё это укладываются в цикл непрерывной интеграции и доставки – Continuous Integration/Continuous Delivery (CI/CD). Как раз именно здесь возникает понятие DevSecOps.

Примеры распределения инструментов и способов верификации по циклу DevSecOps [7]:



Выводы

- Безопасность приложений является отдельным направлением в области обеспечения информационной безопасности.
- Существуют различные подходы к обеспечению безопасности приложений, имеющие свои акценты.
- Тематика безопасности приложений не ограничивается только практиками безопасного кодирования (secure coding practices).
- Существуют различные способы верификации, требующие соответствующих *своих* инструментов.
- Меры обеспечения безопасности приложения, а также другие меры защиты в процессе разработки ПО должны быть применены как можно раньше, а после этого должны применяться на непрерывной основе, т.е. должен быть сформирован цикл DevSecOps.

Дополнительные материалы

1. Книга «Certified Secure Software Lifecycle Professional», Артур Конклин, Дэн Шумейкер, 2022.
2. Книга «Семь безопасных информационных технологий», Александр Баронов и др., 2017 г.
3. OWASP. [Secure Coding Practices](#).
4. Статья «[Shift-Left Security: как сделать разработку безопасной и эффективной?](#)», Сергей Володин.
5. Статья «[Как организовать безопасную разработку приложений \(DevSecOps\) в России](#)», Игорь Новиков.

Использованная литература

1. Microsoft Corporation. [Simplified Implementation of the Microsoft SDL](#).
2. NIST. [Secure Software Development Framework](#).
3. ISO/IEC 27034-1:2018 «[Information technology. Security techniques. Application security](#)» (на русском языке [ГОСТ Р ИСО/МЭК 27034-1-2014](#)).

4. ГОСТ Р 56939-2016 «[Защита информации. Разработка безопасного программного обеспечения. Общие требования](#)».
5. Статья «[Shift-Left Security: как сделать разработку безопасной и эффективной?](#)», Сергей Володин.
6. ISO/IEC 27034-3:2018 «[Information technology. Application security. Part 3: Application security management process](#)» (на русском языке [ГОСТ Р ИСО/МЭК 27034-3-2021](#)).
7. [DevSecOps by Swordfish Security](#).
8. [DevSecOps by Gardener](#).